MATH 205

# Matrices as Transformations

LECTURE 13 · MATRICES AS TRANSFORMATIONS

*What if a matrix isn't a grid of numbers to reduce, but a MACHINE — something that takes a vector in, and moves it somewhere new?*

SAJAL SOHAIL RANA

WELCOME BACK

# Starting a New Arc

### Last time

- Null space, nullity, and the dimension theorem
- Row space and column space, read straight off echelon form
- Rank — three different ways to compute the same number

### This Week: Linear Transformations

- Today: matrices as geometric functions — a gallery of shapes
- Next: the formal definition, kernel & range
- Then: one-to-one, onto, inverses
- Finally: building the matrix of any linear transformation

### Course Roadmap

1. Systems of Linear Equations
2. Matrices & Matrix Algebra
3. Determinants
4. Vector Spaces
5. **Linear Transformations**

LECTURE 1 OF 5

*A change in viewpoint: every matrix A you've reduced, inverted, and taken the rank of is ALSO a machine — plug in a vector, get a new one out.*

*Today is the most visual lecture of the course so far — lots of pictures, few long computations.*

7.1 MATRICES AS FUNCTIONS

# A Change in Viewpoint

*So far, Ax = b has been an equation to solve for x. Today, we fix A and let x vary — turning Ax into a function that eats vectors and produces vectors.*

# Same Equation, New Reading

We write it as b = Ax instead of Ax = b — on purpose:

$$b = A\underline{x}$$

| | |
|---|---|
| **INPUT (independent variable)** | x — a vector in $\mathbb{R}^n$ |
| **OUTPUT (dependent variable)** | b — a vector in $\mathbb{R}^m$ |
| **THE FUNCTION** | A — fixed; it's what does the transforming |

*As x varies over $\mathbb{R}^n$, b = Ax traces out where A "sends" every possible input. That's the picture we build today.*

# Domain, Codomain, Range

A transformation T from $\mathbb{R}^n$ to $\mathbb{R}^m$ is a rule that assigns to each vector x in $\mathbb{R}^n$ a vector T(x) in $\mathbb{R}^m$.

We write $T : \mathbb{R}^n \to \mathbb{R}^m$.

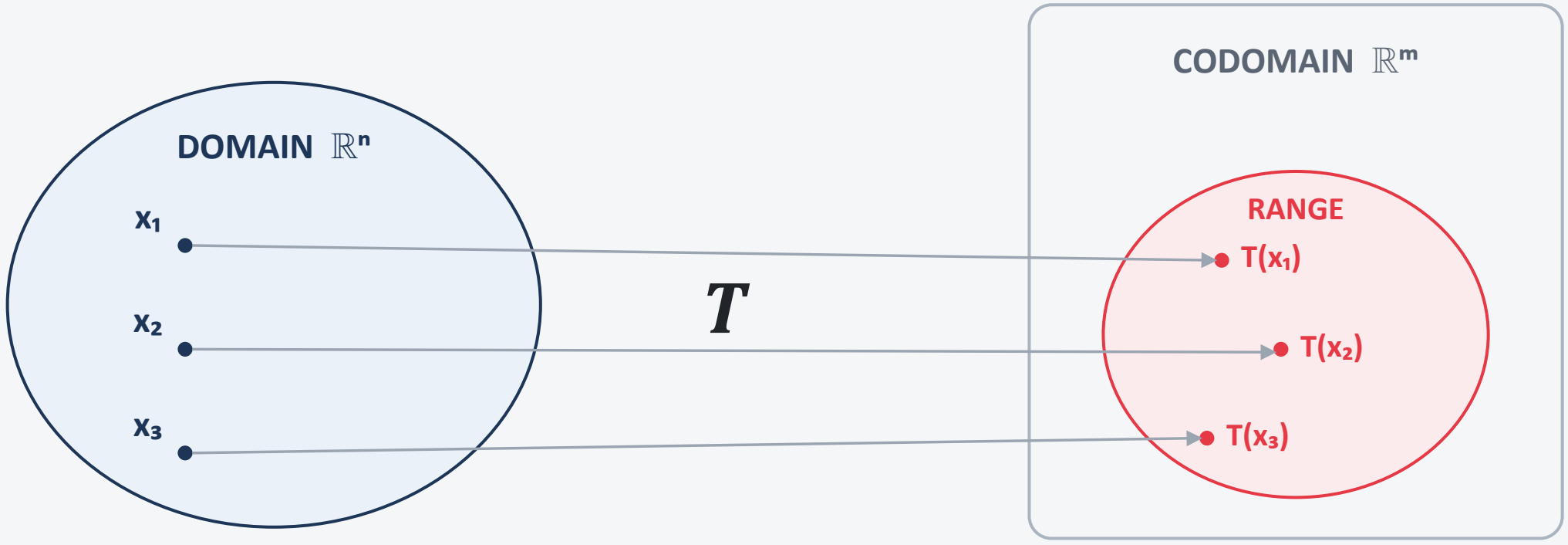
| | |
|---|---|
| **DOMAIN** | $\mathbb{R}^n$ — the space T's inputs live in |
| **CODOMAIN** | $\mathbb{R}^m$ — the space T's outputs are DECLARED to live in |
| **RANGE** | the set of outputs T actually produces — may be smaller than the codomain |

*Codomain is where outputs COULD live. Range is where they ACTUALLY land — that gap is exactly what "onto" will measure next lecture.*

PICTURE

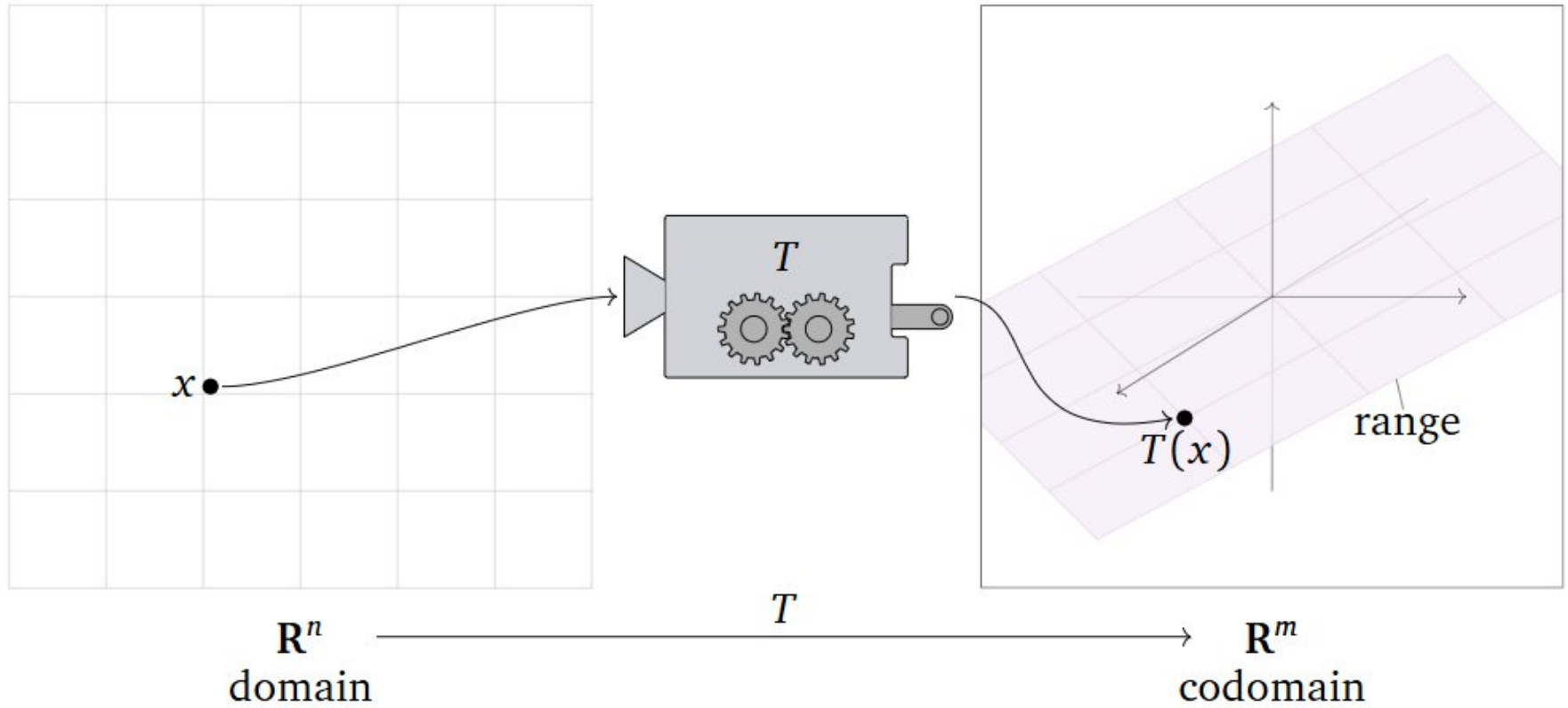
# Seeing a Transformation

Think of T as a machine: vectors go in on the left, vectors come out on the right.




*Every point in the domain has exactly one arrow out.*
*The range (coral) can be a small piece of the codomain — that gap is normal.*

It may help to think of $T$ as a "machine" that takes $x$ as an input, and gives you $T(x)$ as the output.




The points of the domain $\mathbf{R}^n$ are the *inputs* of $T$: this simply means that it makes sense to evaluate $T$ on vectors with $n$ entries, i.e., lists of $n$ numbers. Likewise, the points of the codomain $\mathbf{R}^m$ are the *outputs* of $T$: this means that the result of evaluating $T$ is always a vector with $m$ entries.

7.2 A GALLERY OF TRANSFORMATIONS

# Shapes Worth Knowing

*Projection, reflection, dilation, rotation, and shear — every one of these is a 2×2 or 3×3 matrix.*

*For each, we'll track what happens to $e_1$ and $e_2$: that's all a matrix transformation needs to remember.*

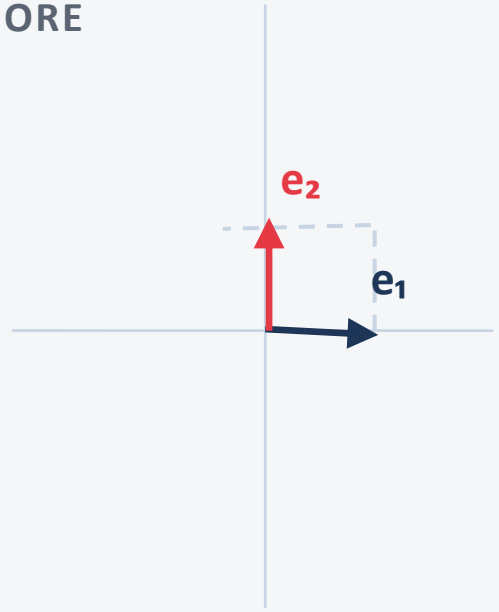
GALLERY

# Projection onto the x-axis

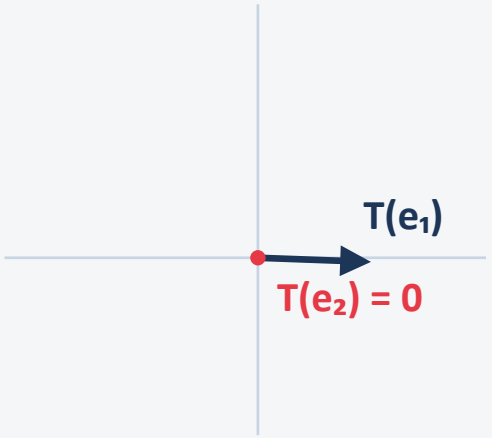
$$A = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

Squashes every point straight down onto the x-axis — the y-coordinate is thrown away.

BEFORE



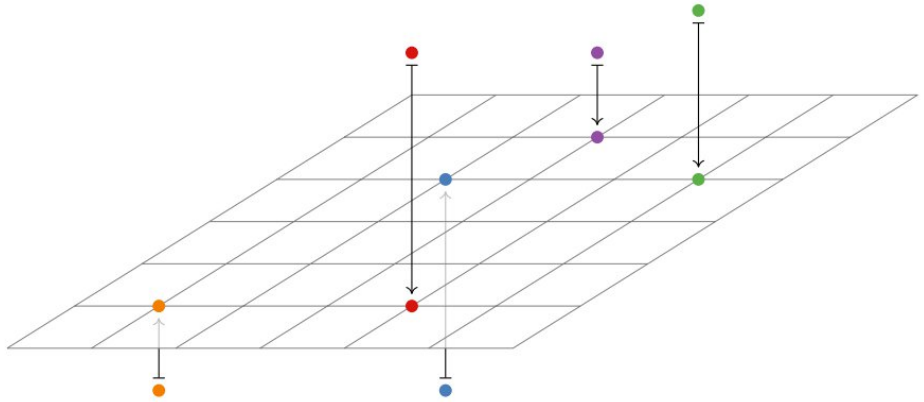

AFTER



GALLERY

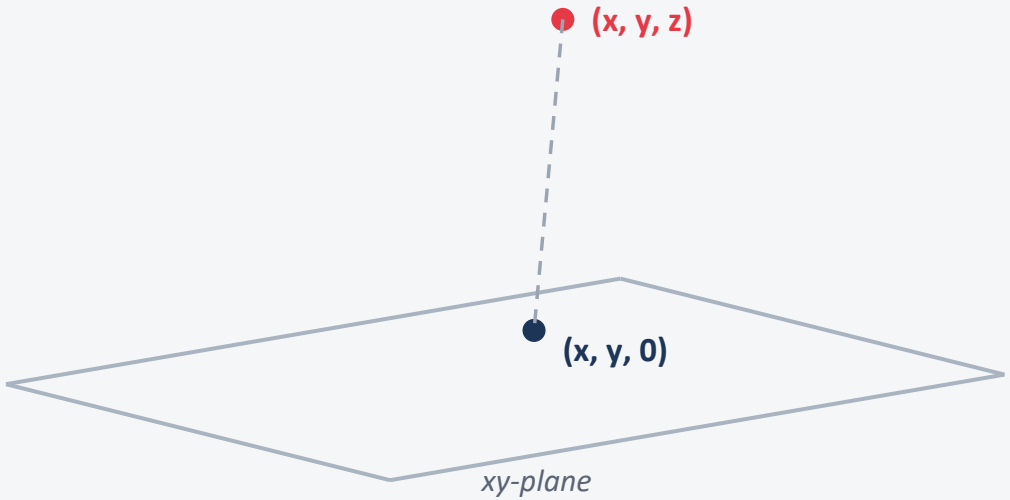
# Projection onto the xy-plane

Multiplication by $A$ simply sets the $z$-coordinate equal to zero: it *projects vertically onto the $xy$-plane*.



$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

In $\mathbb{R}^3$: keep x and y, set z to 0.
Every point drops straight down to its "shadow" on the floor.




See example on Pg 116: Projection onto the xy plane

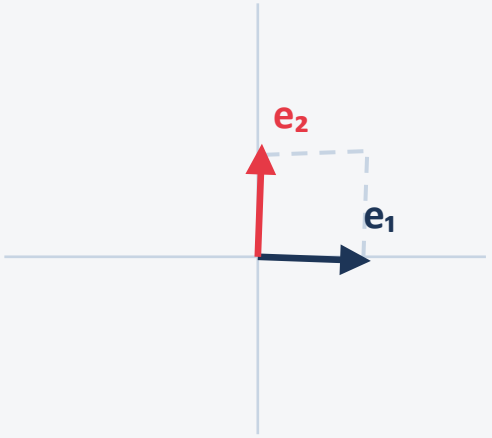
GALLERY

# Reflection About the x-axis

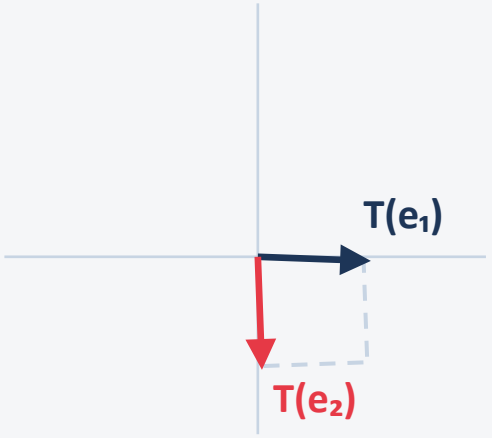
$$A = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

The x-axis stays fixed; everything above it flips to the same distance below, and vice versa.

BEFORE




AFTER




See example on Pg 116-117: Reflection across the y-axis

GALLERY

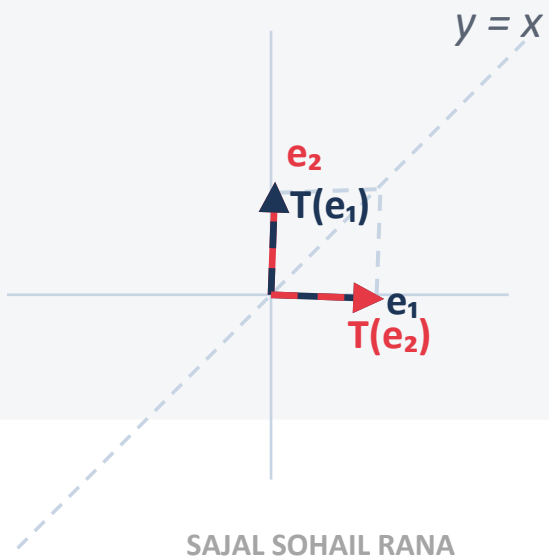
# Reflection About the Line y = x

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Swaps the coordinates: (x, y) ↦ (y, x).

Points on the line y = x don't move at all.

*Solid arrows: before ($e_1$, $e_2$). Dashed arrows: after ($T(e_1)$, $T(e_2)$) — they've swapped places across the mirror line.*



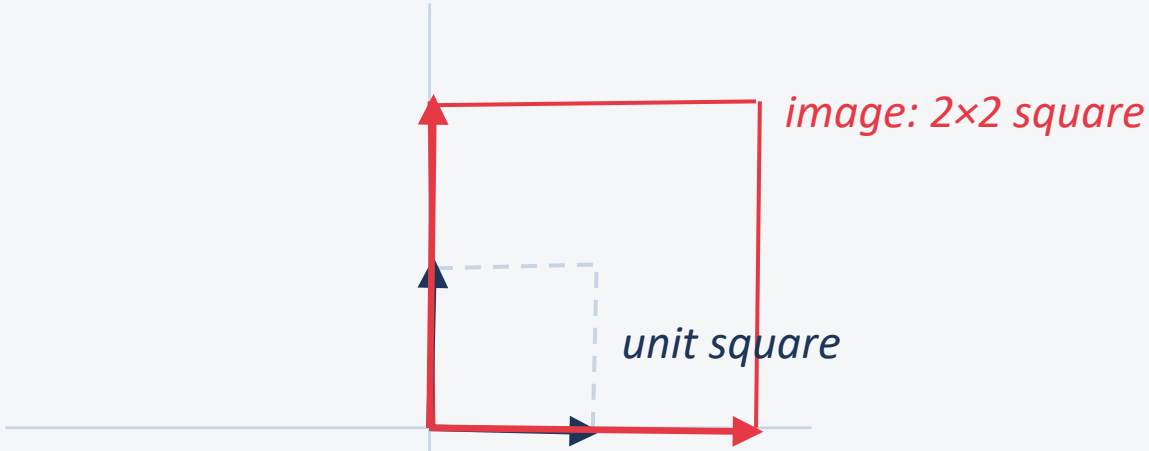
GALLERY

# Dilation (Scaling)

$$A = \begin{bmatrix} k & 0 \\ 0 & k \end{bmatrix}$$

Every vector stretches (or shrinks) by the same factor k, in every direction. k > 1 expands; 0 < k < 1 contracts.

**k = 2**




See example on Pg 117: Dilation by factor of 1.5

GALLERY

# The Identity Transformation

The identity transformation $I : \mathbb{R}^n \to \mathbb{R}^n$ leaves every vector exactly where it is.

$$\mathbf{I}(\underline{x}) = \underline{x} \quad \text{for all } \underline{x} \in R^n$$

Its matrix is the identity matrix $I_n$ — 1's down the diagonal, 0's everywhere else:

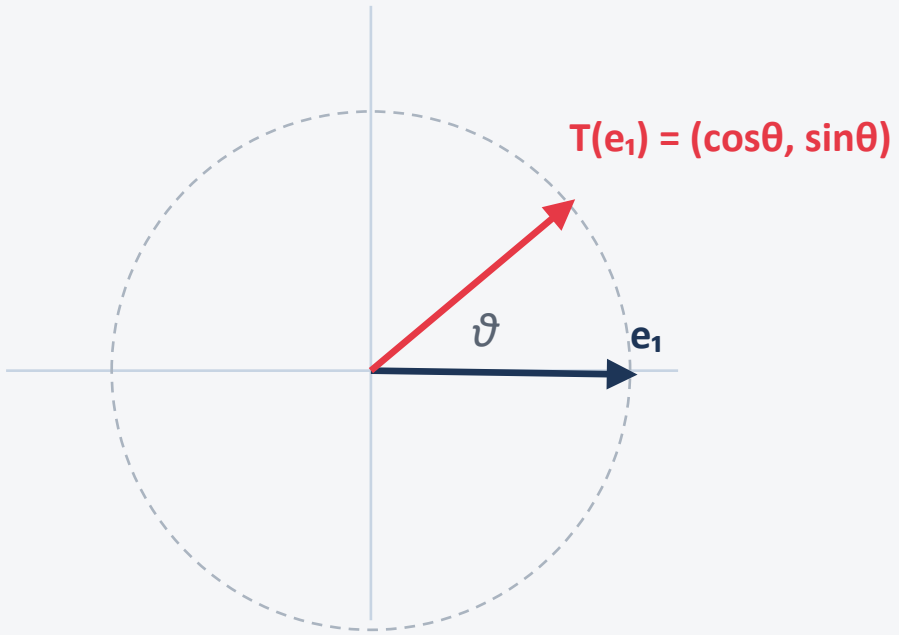
$$I_n = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}$$

See example on Pg 118: 2x2 identity matrix

GALLERY

# Rotation — Deriving the Matrix

Rotate every vector counterclockwise by angle θ. Track $e_1$ and $e_2$ around the unit circle:




By similar reasoning, $e_2 = (0,1)$ rotates to $(-\sin\theta, \cos\theta)$. These images become the COLUMNS of the matrix:

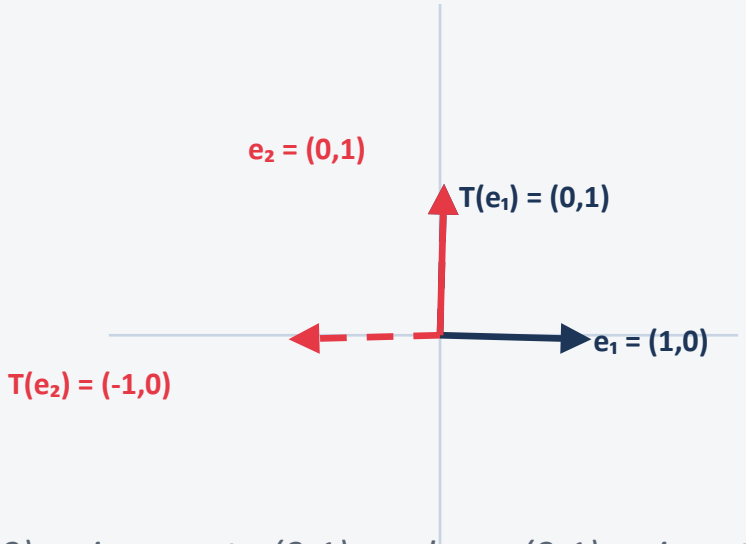
$$T(\underline{e}_1) = (\cos\theta, \sin\theta), \qquad T(\underline{e}_2) = (-\sin\theta, \cos\theta)$$

$$A = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

WORKED EXAMPLE

# Rotating by 90°

$$A = \begin{bmatrix} \cos 90^\circ & -\sin 90^\circ \\ \sin 90^\circ & \cos 90^\circ \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$




*cos 90° = 0 and sin 90° = 1, so $e_1 = (1,0)$ swings up to (0,1), and $e_2 = (0,1)$ swings to (−1,0). A full quarter-turn, every time.*

See example on Pg 118-119: Rotation

GALLERY

# Shear

$$A = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$$

$e_1$ stays put; $e_2$ tips over by k units.

A square leans into a parallelogram — think of pushing the top of a deck of cards.

**k = 1**




See example on Pg 119: shear

TRY IT YOURSELF

# Name That Transformation

*Match each matrix to what it does — cover the answer and guess first.*

| | | |
|---|---|---|
| 1 | $\begin{bmatrix}0&1\\1&0\end{bmatrix}$ | *Reflection about y = x* |
| 2 | $\begin{bmatrix}3&0\\0&3\end{bmatrix}$ | *Dilation by a factor of 3* |
| 3 | $\begin{bmatrix}1&0\\0&0\end{bmatrix}$ | *Projection onto the x-axis* |
| 4 | $\begin{bmatrix}1&2\\0&1\end{bmatrix}$ | *Shear (k = 2)* |

*Notice the pattern: reading off what happens to $e_1$ (column 1) and $e_2$ (column 2) tells you the whole story.*

RECAP

# The Gallery, All Together

### Projection

onto x-axis: [[1,0],[0,0]] — squashes one dimension flat.

### Reflection

about a line: swaps or flips coordinates across it.

### Dilation

[[k,0],[0,k]] — stretches or shrinks uniformly.

### Rotation

[[cosθ,-sinθ],[sinθ,cosθ]] — spins by angle θ.

### Shear

[[1,k],[0,1]] — tips one axis over, leaves the other fixed.

### Identity

$I_n$ — leaves every vector exactly where it started.

APPLICATION

# Where This Shows Up: Computer Graphics

Every time a video game rotates a character, a phone pinch-zooms a photo, or a CAD tool mirrors a part — it's one of today's matrices, applied to every vertex of the shape.

| | |
|---|---|
| **Rotation matrices** | spin 3D models in games and CAD software, frame by frame. |
| **Dilation** | pinch-to-zoom, and scaling sprites up or down without redrawing them. |
| **Reflection** | Mirroring a character **sprite** (a 2D image or animation frame used in a video game) means **horizontally flipping the graphic so it faces the opposite direction**. |
| **Shear** | italic text rendering, and “leaning” skewed scanned documents. |

*Composing several of these (rotate, then scale, then shear) is exactly matrix multiplication — the subject of a later lecture.*

7.3 RANGE = COLUMN SPACE

# A Bridge to What You Know

*You already know how to find a column space — you built bases for them two lectures ago.*

*It turns out that IS the range of the matrix transformation.*

# Why Range = Column Space

$$\text{Range}(T) = \{\, T(\underline{x}) \mid \underline{x} \in R^n \,\}$$

Write A by its columns, A = [$\mathbf{v_1}$ $\mathbf{v_2}$ ⋯ $\mathbf{v_n}$]. Then for any x:

$$T(\underline{x}) = A\underline{x}$$

*= $x_1v_1 + x_2v_2 + \cdots + x_nv_n$ — a linear combination of A's columns.*

So as x ranges over every possible input, T(x) = Ax ranges over every possible LINEAR COMBINATION of the columns of A — exactly the column space.

$$\text{Range}(T) = \text{Col}(A)$$

WORKED EXAMPLE

# Finding a Range

Let T(x) = Ax where:

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

Column 2 is a multiple of column 1 — (2,4) = 2·(1,2) — so the columns are dependent, and the column space collapses to a single line.

$$\text{Range}(T) = \text{Span}\left\{ \begin{bmatrix} 1 \\ 2 \end{bmatrix} \right\}$$

*No matter what x you plug in, Ax always lands somewhere on the line through (1,2). The codomain is $\mathbb{R}^2$, but the range is just this one line — T is far from onto.*

CONTRAST

# Not Every Transformation Is a Matrix (see pp.122 - 123)

A robot arm has three joints, rotated by angles $\theta_1$, $\theta_2$, $\theta_3$. Define $T(\theta_1,\theta_2,\theta_3)$ = the position of the hand.

This T is a perfectly good transformation — one input, one output — but it involves sines and cosines of the angles in complicated, tangled-up ways. There is NO matrix A with T(x) = Ax.

*(This kind of problem is called inverse kinematics — it needs tools well beyond linear algebra.)*

What made today's transformations special is that EACH coordinate of the output is a linear expression in the inputs — no products of variables, no sines of x, no constants added on. That's exactly the property we formalize next lecture.

RECAP

# Key Takeaways

### Function View

b = Ax: fix A, vary x — a matrix is a machine, not just an equation.

### Domain / Codomain

$\mathbb{R}^n$ and $\mathbb{R}^m$ — where inputs live and where outputs are declared to live.

### Range

the outputs T actually hits — can be smaller than the codomain.

### The Gallery

projection, reflection, dilation, rotation, shear — each a simple 2×2 or 3×3 matrix.

### Columns = Images of $e_i$

a matrix's columns are exactly what it does to the standard basis vectors.

### Range = Col(A)

the range of T(x)=Ax is precisely the column space of A.

NEXT TIME

# Looking Ahead

### Before Next Lecture

**Sketch it yourself**

Pick your own 2×2 matrix, find $T(e_1)$ and $T(e_2)$, and sketch what happens to the unit square. Bring a surprising one to class.

### A way to check!

*If a matrix has a column of zeros, that direction always maps to 0 — a fast way to sanity-check a picture.*

Lecture 14

## Linear Transformations, Formally

- The formal definition: $T(u+v)=T(u)+T(v)$ and $T(ku)=kT(u)$.
- Proving a transformation is (or isn't) linear — beyond just matrices.
- Kernel and range, revisited with worked examples.

*Reference: Anton & Rorres, Ch. 8 · ILA §3.1–3.3*

# Questions?

*Matrices as Transformations · Lecture 13 complete*

**NEXT: LINEAR TRANSFORMATIONS, FORMALLY**